CAHIER DES CHARGES

NetOps AI — Plateforme de Network Automation, Observability & AIOps

Version 1.0 • Statut : projet en développement • Environnement : laboratoire GNS3/FRRouting

1. Contexte et problématique

Les équipes d'exploitation réseau doivent superviser la disponibilité des équipements, comprendre les incidents, maîtriser les changements de configuration et réduire le temps de diagnostic. Le projet propose un laboratoire reproductible et une plateforme intégrée d'automatisation et d'observabilité, avec assistance au diagnostic.

2. Objectifs

- Centraliser une topologie et un inventaire cohérents des équipements et interfaces.
- Automatiser la sauvegarde, la vérification et le déploiement contrôlé des configurations.
- Collecter métriques, états OSPF et journaux pour détecter les incidents.
- Corréler les symptômes afin de proposer des causes probables et d'estimer les impacts.
- Présenter les alertes, diagnostics et historiques dans une interface exploitable.
- Documenter, tester et démontrer les fonctionnalités dans des scénarios reproductibles.

3. Périmètre et hypothèses

Inclus

Topologie de départ R1–R2–R3, LAN PC1 et PC2, routage OSPF ; scénarios de panne/rétablissement ; automatisation Ansible ; collecte et supervision Prometheus/Grafana ; journaux ; moteur de diagnostic ; API ; tests et documentation.

Extensions conditionnelles

BGP, VLAN/ACL, NetBox, validation Batfish, modèles scikit-learn et assistant LLM sont envisagés uniquement après validation de la faisabilité et des ressources. Les outils ne sont pas ajoutés uniquement pour multiplier les technologies.

Hors périmètre initial

Production sur équipements réels, remédiation autonome sans approbation, garantie de diagnostic parfait et haute disponibilité industrielle.

4. Exigences fonctionnelles

ID	Exigence	Critère d'acceptation
F01	Découvrir et référencer les équipements	Inventaire R1/R2/R3, IP, interfaces et liens cohérents.
F02	Collecter les états réseau	Interfaces, voisins OSPF, routes et disponibilité accessibles.
F03	Automatiser les configurations	Playbook idempotent, sauvegarde préalable et rapport de changements.

F04	Superviser en continu	Métriques horodatées et dashboard opérationnel.
F05	Détecter les incidents	Alerte sur perte d'adjacence R2–R3 dans un délai mesuré.
F06	Corréler les symptômes	Rapport expliquant interfaces UP/DOWN, OSPF et connectivité.
F07	Historiser les événements	Début, fin, sévérité et équipement concerné consultables.
F08	Recommander une action	Suggestion justifiée, non exécutée sans approbation.
F09	Exposer une API	Endpoints documentés pour états et incidents.
F10	Démontrer le rétablissement	Retour à la normale visible et incident clôturé.

5. Exigences non fonctionnelles

- Sécurité : secrets hors du dépôt, accès SSH contrôlé, permissions minimales.
- Traçabilité : horodatage, logs d'exécution et historique des modifications.
- Maintenabilité : composants modulaires, configuration externe et documentation.
- Fiabilité : distinguer données indisponibles, tests impossibles et pannes réelles.
- Reproductibilité : procédures d'installation et tests automatisés.
- Performance : fréquence de collecte adaptée aux ressources ; mesurer la latence d'alerte.

6. Architecture et choix technologiques

Couche	Technologies envisagées	Responsabilité
Laboratoire	GNS3, Docker, FRRouting	Topologie, routage et scénarios d'incident
Source de vérité	NetBox	Inventaire, interfaces, IP et liens
Automatisation	Ansible, Netmiko, Jinja2, Git	Configuration et contrôle des changements
Métriques	Prometheus, exporters	Collecte de séries temporelles
Logs	Loki, Fluent Bit	Centralisation des événements
Visualisation/alertes	Grafana, Alertmanager	Dashboards et notifications
Analyse	Python, NetworkX, règles ; scikit-learn optionnel	Corrélation, impact et anomalies
API et données	FastAPI, PostgreSQL	Services et historique métier
Livraison	Docker Compose, pytest, GitHub Actions	Déploiement, tests et CI

Contraintes d'intégration : FRRouting doit exposer les données par CLI, API ou exporter adapté ; Netmiko exige SSH sur les routeurs ; le serveur de supervision peut nécessiter un hébergement séparé de la GNS3 VM.

7. État initial déjà validé

- Adjacences OSPF R1–R2 et R2–R3 FULL et routes inter-LAN apprises.
- Collecte CLI FRRouting automatisée et export JSON fonctionnels.
- Détection d'une panne simulée R2–R3 puis du rétablissement.
- Analyse croisée interfaces/OSPF/ping et export de rapport.

- Dashboard Streamlit initial exécuté.

Dette technique identifiée : identifiants Docker statiques, sorties CLI partiellement non structurées, déclenchement manuel de la collecte, persistance des configurations FRR à fiabiliser, pas encore de stockage historique robuste.

8. Phases, livrables et estimation

Phase	Livrables	Estimation
P1	SSH FRR, sauvegarde persistante, inventaire, playbooks Ansible	1–2 semaines
P2	Prometheus/exporters, Grafana, alerting, logs essentiels	1–2 semaines
P3	NetBox, API, historique et corrélation des incidents	2 semaines
P4	Diagnostic avancé, NetworkX, démonstration de remédiation contrôlée	1–2 semaines
P5	Tests, CI/CD, documentation, présentation	1 semaine

Estimation indicative : MVP 3–4 semaines ; version portfolio PFE 6–8 semaines ; AIOps avancé 10–14 semaines, à raison de 15–20 h/semaine. La durée dépend fortement des intégrations et du matériel disponible.

9. Scénarios de recette

Test	Procédure	Résultat attendu
T01	Réseau sain	Toutes les adjacences attendues FULL, aucune alerte active.
T02	Suspendre R2–R3	Perte OSPF signalée, interface éventuellement UP, impact identifié.
T03	Rétablir R2–R3	Alerte résolue et historique conservé.
T04	Modifier une configuration en mode contrôlé	Diff, validation, sauvegarde et traçabilité.
T05	Exporter/consulter un incident	API et dashboard concordants avec l'historique.
T06	Collecte impossible	État UNKNOWN/UNAVAILABLE, sans faux diagnostic certain.

10. Risques et mesures

Risque	Mesure
Perte de configuration FRR	Fiabiliser la persistance avant redémarrage ; sauvegardes hors conteneur.
Ressources VM limitées	Séparer les services, limiter la rétention, mesurer CPU/RAM.
Accès SSH/exports incompatibles	Valider la connectivité et choisir des adaptateurs compatibles FRR.
Complexité excessive	Prioriser le chemin critique et rendre les extensions

	optionnelles.
Faux positifs / IA non fiable	Évaluer sur incidents labellisés ; validation humaine obligatoire.

11. Critères de clôture

- Démonstration complète : sain → panne → alerte → diagnostic → rétablissement → historique.
- Au moins un workflow Ansible de configuration/contrôle réellement opérationnel.
- Dashboards et alertes alimentés par des données mesurées, non simulées dans l'interface.
- Documentation d'installation, architecture, limites et scénarios de test disponibles.
- Dépôt Git nettoyé ; aucun secret exposé ; tests reproductibles.